AI Assisted Coding

Assignment 16.3

Name: Vivek

Hallticket:2303A52009

ask 1 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
• List all appointments for a specific doctor.
• Retrieve patient history by patient ID.
• Count total patients treated by each doctor.
Expected Output:
• Normalized schema and SQL queries with joins.

```sql
---create a table for • Tables: Doctors, Patients, Appointments.
Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
• List all appointments for a specific doctor.
• Retrieve patient history by patient ID.
• Count total patients treated by each doctor.---
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Appointments;
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    specialty TEXT NOT NULL
);
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    date_of_birth DATE NOT NULL
);
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
```

SQL ▾ 1 / 1 1 - 4 of 4

appointment_id	doctor_name	appointment_date
1	Dr. Smith	2024-07-01
2	Dr. Johnson	2024-07-02
3	Dr. Lee	2024-07-03
4	Dr. Smith	2024-07-04

SQL ▾ 1 / 1 1 - 2 of 2

appointment_id	patient_name	appointment_date
1	John Doe	2024-07-01
4	Jane Doe	2024-07-04

Task 02 :

lab19.py 9+, U | lab_18.py U | database.db U | students.sql U | hospital.sql U | SQLite

SQL > hospital.sql

```sql
--- Task 2 – Real-Time Application: Online Shopping Database
---Scenario:
-- Design a database for an E-commerce platform.
-- Requirements:
-- • Tables: Users, Products, Orders, OrderDetails.
-- • Generate queries to:
-- • Retrieve all orders by a user.
-- • Find the most purchased product.
-- • Calculate total revenue in a given month.
-- • AI should also suggest normalization improvements and query
-- optimization.
-- Expected Output:
-- • Complete schema with relationships + SQL queries for analytics.---
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE
);
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    price REAL NOT NULL
);
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Insert sample data into Users table
INSERT INTO Users (user_id, username, email) VALUES
(1, 'john_doe', 'john.doe@example.com');
-- Insert sample data into Products table
```

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 0 of 0

PROBLEMS 100 | OUTPUT | DEBUG CONSOLE | TERMINAL 11 | PORTS | POSTMAN CONSOLE

TERMINAL

PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>

Ln 89, Col 3 (704 selected) | Spaces: 4 | UTF-8 | CRLF | {} SQLite | SQLite: hospital.db | Go Live

Search | ENG IN | 10:22 AM 08-04-2026

hospital.sql

```sql
CREATE TABLE OrderDetails (
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Insert sample data into Users table
INSERT INTO Users (user_id, username, email) VALUES
(1, 'john_doe', 'john.doe@example.com');
-- Insert sample data into Products table
INSERT INTO Products (product_id, name, price) VALUES
(1, 'Laptop', 999.99),
(2, 'Smartphone', 499.99);
-- Insert sample data into Orders table
INSERT INTO Orders (order_id, user_id, order_date) VALUES
(1, 1, '2024-07-01');
-- Insert sample data into OrderDetails table
INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity) VALUES
(1, 1, 1, 1),
(2, 1, 2, 2);
-- Query to retrieve all orders by a user (e.g., user_id = 1)
SELECT o.order_id, o.order_date, p.name AS product_name, od.quantity
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1;
-- Query to find the most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC
LIMIT 1;
```

SQL ▾ 1 / 1 1 - 2 of 2

order_id	order_date	product_name	quantity
1	2024-07-01	Laptop	1
1	2024-07-01	Smartphone	2

PROBLEMS 106 OUTPUT DEBUG CONSOLE TERMINAL 11 PORTS POSTMAN CONSOLE

TERMINAL

```
PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>
```

UNIQUE constraint failed: Users.u

Source: SQLite

Ln 105, Col 1 (204 selected) Spaces: 4 UTF-8 CRLF { } SQLite ENG

SQL > hospital.sql

```sql
CREATE TABLE OrderDetails (
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
-- Insert sample data into Users table
INSERT INTO Users (user_id, username, email) VALUES
(1, 'john_doe', 'john.doe@example.com');
-- Insert sample data into Products table
INSERT INTO Products (product_id, name, price) VALUES
(1, 'Laptop', 999.99),
(2, 'Smartphone', 499.99);
-- Insert sample data into Orders table
INSERT INTO Orders (order_id, user_id, order_date) VALUES
(1, 1, '2024-07-01');
-- Insert sample data into OrderDetails table
INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity) VALUES
(1, 1, 1, 1),
(2, 1, 2, 2);
-- Query to retrieve all orders by a user (e.g., user_id = 1)
SELECT o.order_id, o.order_date, p.name AS product_name, od.quantity
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1;
-- Query to find the most purchased product
SELECT p.name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_quantity DESC
LIMIT 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

name	total_quantity
Smartphone	2

Task_03

```sql
CREATE TABLE Members (
);
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
-- Insert sample data into Books table
INSERT INTO Books (book_id, title, author, published_date) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10'),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11');
-- Insert sample data into Members table
INSERT INTO Members (member_id, name, email) VALUES
(1, 'Alice Smith', 'alice.smith@example.com');
-- Insert sample data into Loans table
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2024-06-01', NULL);
-- Query to retrieve all books currently issued
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
-- Query to find overdue books (loan date > 30 days)
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND l.loan_date < DATE('now', '-30 days');
-- Query to count number of books loaned by each member
SELECT m.name AS member_name, COUNT(l.loan_id) AS total_loans
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id;
```

SQL ▾ 1 / 1 1 - 1 of 1

title	member_name	loan_date
The Great Gatsby	Alice Smith	2024-06-01

UNIQUE constraint failed: Books.book_id
Source: SQLite
Show output

UNIQUE constraint failed: Users.user_id
Source: SQLite
Show output

PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>


```sql
CREATE TABLE Loans (
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
-- Insert sample data into Books table
INSERT INTO Books (book_id, title, author, published_date) VALUES
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10'),
(2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11');
-- Insert sample data into Members table
INSERT INTO Members (member_id, name, email) VALUES
(1, 'Alice Smith', 'alice.smith@example.com');
-- Insert sample data into Loans table
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES
(1, 1, 1, '2024-06-01', NULL);
-- Query to retrieve all books currently issued
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
-- Query to find overdue books (loan date > 30 days)
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND l.loan_date < DATE('now', '-30 days');
-- Query to count number of books loaned by each member
SELECT m.name AS member_name, COUNT(l.loan_id) AS total_loans
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id;
```


SQL ▾ 1 / 1 1 - 1 of 1

member_name	total_loans
Alice Smith	1

UNIQUE constraint failed: Books.b

Task_5

hospital.sql

```sql
CREATE TABLE Registrations (
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
-- Insert sample data into Students table
INSERT INTO Students (student_id, name, email) VALUES
(1, 'Alice Johnson', 'alice.johnson@example.com');
-- Insert sample data into Faculty table
INSERT INTO Faculty (faculty_id, name, department) VALUES
(1, 'Dr. Smith', 'Computer Science');
-- Insert sample data into Courses table
INSERT INTO Courses (course_id, name, faculty_id) VALUES
(1, 'Database Systems', 1),
(2, 'Operating Systems', 1);
-- Insert sample data into Registrations table
INSERT INTO Registrations (registration_id, student_id, course_id) VALUES
(1, 1, 1),
(2, 1, 2);
-- Query to list all students enrolled in a specific course (e.g., course_id = 1)
SELECT s.name AS student_name, s.email
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
WHERE r.course_id = 1;
-- Query to find faculty members teaching more than 2 courses
SELECT f.name AS faculty_name, COUNT(c.course_id) AS total_courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;
-- Query to retrieve courses with the highest number of registrations
SELECT c.name AS course_name, COUNT(r.registration_id) AS total_registrations
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY total_registrations DESC
LIMIT 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

student_name	email
Alice Johnson	alice.johnson@example.com

UNIQUE constraint failed: Books.book_id

Source: SQLite

Show o


```sql
CREATE TABLE Registrations (
);
-- Insert sample data into Students table
INSERT INTO Students (student_id, name, email) VALUES
(1, 'Alice Johnson', 'alice.johnson@example.com');
-- Insert sample data into Faculty table
INSERT INTO Faculty (faculty_id, name, department) VALUES
(1, 'Dr. Smith', 'Computer Science');
-- Insert sample data into Courses table
INSERT INTO Courses (course_id, name, faculty_id) VALUES
(1, 'Database Systems', 1),
(2, 'Operating Systems', 1);
-- Insert sample data into Registrations table
INSERT INTO Registrations (registration_id, student_id, course_id) VALUES
(1, 1, 1),
(2, 1, 2);
-- Query to list all students enrolled in a specific course (e.g., course_id = 1)
SELECT s.name AS student_name, s.email
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
WHERE r.course_id = 1;
-- Query to find faculty members teaching more than 2 courses
SELECT f.name AS faculty_name, COUNT(c.course_id) AS total_courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;
-- Query to retrieve courses with the highest number of registrations
SELECT c.name AS course_name, COUNT(r.registration_id) AS total_registrations
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY total_registrations DESC
LIMIT 1;
```


SQL ▾ 1 / 1 1 - 1 of 1

course_name	total_registrations
Database Systems	1

UNIQUE constraint failed: Books.book_id

Source: SQLite

Show output

UNIQUE constraint failed: Users.user_id

Source: SQLite

Show output

PROBLEMS 106 OUTPUT DEBUG CONSOLE TERMINAL 11 PORTS POSTMAN CONSOLE

TERMINAL

PS C:\Users\Vivek\OneDrive\Desktop\3-2\AI-Assisted-Coding>

Ln 234, Col 1 (213 selected) Spaces: 4 UTF-8 CRLF SQLite SQLite: hospital.db Go Live